



ATMIYA University, Rajkot

Faculty of Science

Department of Computer Application

B.C.A. / B.SC. I.T

Object Oriented PHP & Laravel

Unit -1 Object Oriented Programming in PHP

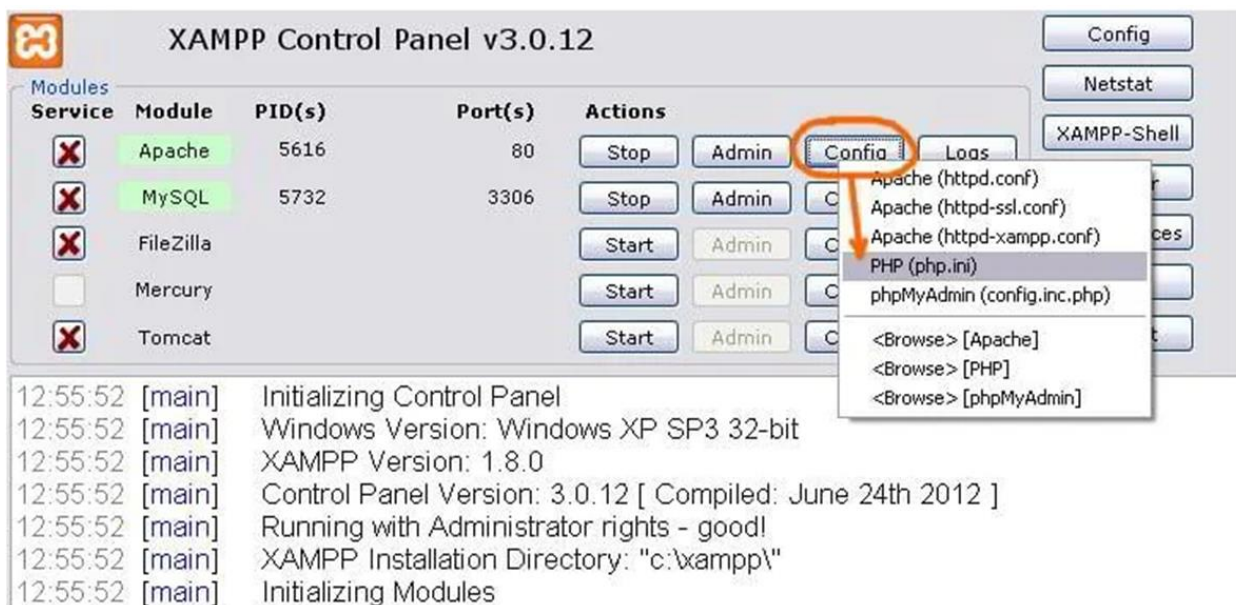
The Basics of PHP and Introduction

1. Introduction, Web Server (xampp Server, Apache Server)

- What is XAMPP Server ?
 - For Local Development, you need to install XAMPP.
 - XAMPP is a local web development environment.
 - You can use XAMPP on your desktop to Design, Develop and Test websites.
 - XAMPP is an application which allows you to setup a webserver and database server instantly without any additional configuration.
 - It is very useful for beginners who want to learn how to make projects(Web Application) using PHP, Perl, MySQL Database.
- What is Apache Server?
 - Apache is the most widely used web server software developed and maintained by Apache Software Foundation.
 - It is free. Develop and provide Open source software.
 - If you host a website, chances are good that you are running either Apache or IIS.
 - These two are the most common web server platforms.
 - Apache combined with MYSQL database and PHP scripting language.

2. Installation and Configuration of PHP (php.ini, httpd.conf)

- **php.ini**
 - PHP.INI file is a configuration file and controls some of the settings of the PHP interpreter.
 - The configuration file is read when PHP starts.
 - The php.ini file is the default configuration file for running applications that require PHP. It is used to control variables such as upload sizes, file timeouts, and resource limits.



- Basic important settings of php.ini file:
- short_open_tag = Off:
- Short open tags look like this: <? ?>. It is OFF by default so you must have to write the full name of the PHP tag.
- **Resource Limit:**
- It indicates the maximum amount of memory a script may consume. Default value is 128 MB.
- **max_execution_time:** maximum execution time of each script, in seconds. Default value is set to 30 sec.
- **Data Handling:**
- register_globals: By default is set to "OFF".
- **httpd.conf**
 - HTTPD stands for Hypertext Transfer Protocol daemon (i.e., Web server). It is a software that runs in the background and waits for incoming requests,
 - The httpd.conf file defines configuration settings for the HTTP protocol and for the operation of the server. It is only accessible by a system administrator

Object Oriented Programming in PHP

1. Defining Classes and Properties, Methods

- We first need to understand the concept of object-oriented programming, also known as OOPs, before learning about PHP classes. Unlike other programming languages like **C++**, **JAVA**, etc., PHP also supports the concept of object-oriented programming.
- **Class**: The user-defined data type combines variables, local data, and functions.
- **How to define Class**:

```
<?php
class demo {
    // code goes here...
}
?>
```

- **Properties** : Class member variables are called properties.
- **Methods**: These are none other than functions defined inside a class and are generally used to access data objects.

2. Access controllers (public, private)

- Properties and methods can have access modifiers which control where they can be accessed.
- There are three access modifiers:
- **public** - the property or method can be accessed from everywhere. This is default
- **protected** - the property or method can be accessed within the class and by classes derived from that class
- **private** - the property or method can ONLY be accessed within the class

3. Instantiating an object, changing values of properties

- **Object**: Classes are nothing without objects! We can create multiple objects from a class.
 - Each object has all the properties and methods defined in the class, but they will have different property values.
 - Objects of a class are created using the **new** keyword.
- **How to define Object**:

```
<?php
class demo {
    // code goes here...
    public $name;
}
$obj= new demo();
?>
```

- Changing values of properties:

- To change the value of property there are two ways:
- 1. Inside the class (by adding a method and use \$this):

```
<?php
class demo {
    public $name;
    function set_name($nm) {
        $this->name = $nm;
    }
}
$obj = new demo();
$obj->set_name("My 1st demo in PHP OOP");

echo $obj->name;
?>
```

- 2. Outside the class (by directly changing the property value):

```
<?php
class demo {
    public $name;
}
$obj = new demo();
$obj->name = " My 1st demo in PHP OOP ";

echo $obj->name;
?>
```

4. Accessor methods (setters & getters)

- Getter and setter strategies are utilized when we need to restrict the direct access to the variables by end-users.
- Getters and setters are methods used to define or retrieve the values of variables, normally private ones.
- Just as the name suggests, a getter method is a technique that gets or recovers the value of an object.
- Also, a setter method is a technique that sets the value of an object.
- Ex:

```
<?php
```

```

class Person{

    private $name;

    public function setName($nm)//setter method
    {

        $this->name = $nm;

    }

    public function getName()//getter method
    {

        return $this->name;

    }

}

$obj = new Person();

$obj->setName("OOP PHP");

echo $obj->getName();

?>

```

5. Defining constructor, destructor

- **Constructor:** A constructor allows you to initialize an object's properties upon creation of the object.
- If you create a `__construct()` function, PHP will automatically call this function when you create an object from a class.
- Notice that the construct function starts with two underscores (`__`)!
- Ex:

```

<?php
class Fruit {
    public $name,$a;
    function __construct($nm,$b) {
        $this->name = $nm;
        $this->a=$b;
    }
}

```

```

    }
    function get_name() {
        echo $this->a;
        return $this->name;
    }
}
$obj = new Fruit("Apple",30);
$val= $obj->get_name();
echo $val;
?>

```

- **Destructor:**
- A destructor is called when the object is destructed or the script is stopped or exited.
- If you create a `__destruct()` function, PHP will automatically call this function at the end of the script.
- Notice that the destruct function starts with two underscores (`__`)!
- Ex:

```

<?php
class Fruit {
    public $name;
    function __construct($name) {
        $this->name = $name;
    }
    function __destruct() {
        echo "The fruit is {$this->name}.";
    }
}
$apples = new Fruit("Apple");
?>

```

6. Keywords : new, this, self, final

- **new** : Objects of a class are created using the new keyword.
- **this** : The `$this` keyword refers to the current object, and is only available inside methods.
- **Self**: self operator represents the current class and thus is used to access class variables or static variables because these members belong to a class rather than the object of that class. (ex: with static method)

- **Final:**The final keyword prevents child classes from overriding a method or constant by prefixing the definition with final. If the class itself is being defined final then it cannot be extended.(ex: inheritance ch-2)

7. Scope resolution operator (::), arrow operator

- **Scope resolution operator (::)**
 - The Scope Resolution Operator or in simpler terms, the double colon, is a token that allows access to static, constant, and overridden properties or methods of a class.
 - When referencing these items from outside the class definition, use the name of the class. (ex: with static members)
- **arrow operator (->)**
 - The -> arrow symbol is an access operator that is typically used for any access and call methods and any properties in PHP objects.

8. Static properties & methods

- **Static properties:**
 - Static properties can be called directly - without creating an instance of a class.
 - Static properties are declared with the static keyword:
 - To access a static property use the class name, double colon (::), and the property name:
 - syntax:
static variable_name;
 - Ex:

```
<?php
class demo{
    private static $static_member = "PHPOOP";
    function __construct() {
        echo self::$static_member;
        // Accessing static variable
    }
}
new demo();
?>
```

- **Static Methods:**
 - Static methods can be called directly - without creating an instance of the class first.
 - Static methods are declared with the static keyword:

- To access a static method use the class name, double colon (::), and the method name.

- Syntax for declaration:

```
public static function test()
{
    // Method implementation
}
```

- Syntax for calling method:

```
classname::function_name();
```

- Ex:

```
<?php
class calc{

    static $count;

    public static function getCount() {
        return self::$count++;
    }
}

calc::$count = 1;
for($i = 0; $i < 5; ++$i) {
    echo 'The next value is: '.
    calc::getCount() . "<br>";
}

?>
```

9. Method chaining

- Method Chaining is nothing but the concatenation of multiple methods to increase the readability of code and avoid putting all the code in the single function.

- Ex:

```
<?php
class demo {
    private $str;
    function __construct()
    {
```

```
        $this->str = "I ";
    }
    function addA()
    {
        $this->str .= "am ";
        return $this;
    }
    function addB()
    {
        $this->str .= "Programmer";
        return $this;
    }
    function getStr()
    {
        return $this->str;
    }
}
$obj = new demo();
echo $obj->addA()->addB()->getStr();
?>
```

Unit -2 Features of OOP & Magic Methods & Constants

1. Inheritance, Protected members

- **Inheritance**

- Inheritance in OOP = When a class derives from another class.
- The child class will inherit all the public and protected properties and methods from the parent class. In addition, it can have its own properties and methods.
- An inherited class is defined by using the extends keyword.

- Syntax

class B extends A

- Example

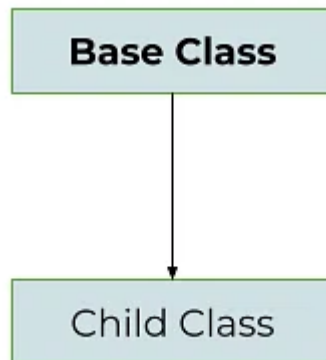
```
<?php
class A
{
    //properties, constants and methods of class A
}
class B extends A
{
    //public and protected methods inherited
}
?>
```

- Ex:

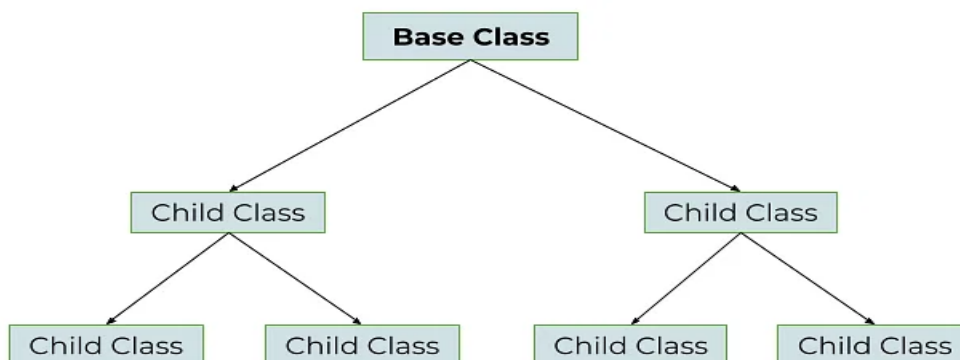
```
<?php
class parentclass{
    public function publicmethod(){
        echo "This is public method of parent class
    ";
    }
    protected function protectedmethod(){
        echo "This is protected method of parent class
    ";
    }
    private function privatemethod(){
        echo "This is private method of parent class
    ";
    }
}
class childclass extends parentclass{
    public function childmethod(){
        $this->protectedmethod();
    }
}
```

```
        // $this->privatemethod(); // this will produce error
    }
}
$obj=new childclass();
$obj->publicmethod();
$obj->childmethod();
?>
```

- PHP offers mainly three types of inheritance based on their functionality. These three types are as follows:
- **Inheritance:** Single inheritance is the most basic or fundamental type of inheritance offered by PHP. There is only one base class and one sub/derived class in a single inheritance and the subclass is directly inherited from the base class.



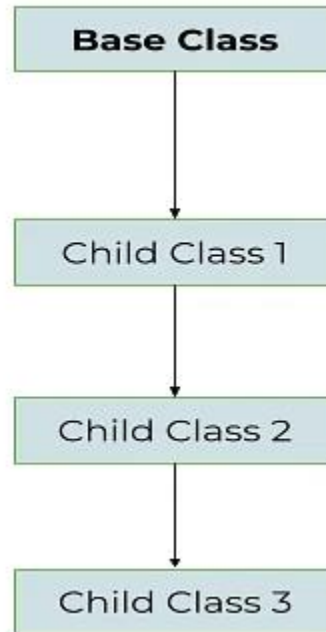
- **Hierarchical Inheritance:** As the name suggests, the hierarchical inheritance adopts a tree-like structure, where multiple derived classes are inherited from the base class.



- **Ex:**

```
<?php
class ParentClass {
    var $var = "This is first var";
    public $fist_name;
    // simple class method
    function returnVar() {
        echo $this->fist_name;
    }
    function set_fist_name($set_this){
        $this->fist_name = $set_this;
    }
}
class child_1 extends ParentClass {
    function setVal($set_this){
        $this->fist_name = $set_this;
    }
    function getVal(){
        echo $this->fist_name;
    }
}
class child_2 extends ParentClass {
    function setVal($set_this){
        $this->fist_name = $set_this." - ".$set_this;;
    }
    function getVal(){
        echo $this->fist_name;
    }
}
$obj1 = new child_1();
$obj1->setVal("This is first child class");
$obj1->getVal();
echo "<br/><br/>";
$obj2 = new child_2();
$obj2->setVal("This is second child class");
$obj2->getVal();
?>
```

- **Multilevel Inheritance:** Multilevel inheritance is the third type of inheritance supported in PHP. It occurs at different levels. Here, one base class is inherited by a derived class, then that derived class is inherited by other derived classes, and so on.



- Ex:

```
<?php
```

```
// base class named "Jewellery"
class Jewellery {
    public function totalCost() {
        return ' total jewellery cost: 600000';
    }
}

// derived class named "Necklace"
// inherited form class "Jewellery"
class Necklace extends Jewellery {
    public function necklace() {
        return ' necklace cost: 450000';
    }
}

// derived class named "Pendant"
// inherited form class "Necklace"
class Pendant extends Necklace {
    public function pendantCost() {
        return ' pendant cost: 600000';
    }

    public function priceList() {
        echo '1. ' . $this->totalCost() . PHP_EOL;
        echo '2. ' . $this->necklace() . PHP_EOL;
    }
}
```

```

        echo '3. ' . $this->pendantCost() . PHP_EOL;
    }
}
// creating object of
// the derived class
$obj = new Pendant();
$obj->priceList();
?>

```

- **Protected members**

- Just like the private access modifier, protected access modifier also restricts the access of class variables and methods outside the class.
- But the protected class variables and functions can be accessed inside the class and inside the subclass(a class that inherits the class).
- The protected keyword ensures that all properties/methods declared/defined using this keyword cannot be accessed externally.
- However, its main advantage over the "private" keyword is that such methods/properties can be inherited by child classes.
- The following example demonstrates a class with Protected members:

```

class derived_class_name extends base_class_name
{
    // protected data member
    protected $var_name;
    // protected member function
    protected function function_name()
    {
        // define the function here
    }
}

```

2. Method overriding

- If a method inherited from the parent class is redefined in the child class, the new definition overrides earlier functionality. In following example, public method is defined again in child class.
- Ex:

```

<?php
class parentclass{
    public function publicmethod(){
        echo "This is public method of parent class" ;
    }
}

```

```

    }
    protected function protectedmethod(){
        echo "This is protected method of parent class" ;
    }
    private function privatemethod(){
        echo "This is private method of parent class" ;
    }
}
class childclass extends parentclass{
    public function publicmethod(){
        echo "public method of parent class is overridden in child class" ;
    }
}
$obj=new childclass();
$obj->publicmethod();

?>

```

3. Final Keyword in inheritance

- The final keyword prevents child classes from overriding a method or constant by prefixing the definition with final.
- If the class itself is being defined final then it cannot be extended.
- **Example #1 Final methods example**

```

<?php
class BaseClass {
    final public function moreTesting() {
        echo "BaseClass::moreTesting() called\n";
    }
}

class ChildClass extends BaseClass {
    public function moreTesting() {
        echo "ChildClass::moreTesting() called\n";
    }
}
// Results in Fatal error: Cannot override final method
BaseClass::moreTesting()

?>

```

- **Example #2 Final class example**

```

<?php
final class BaseClass {
    public function test() {
        echo "BaseClass::test() called\n";
    }
}

```



```

    }

    // As the class is already final, the final keyword is redundant
    final public function moreTesting() {
        echo "BaseClass::moreTesting() called\n";
    }
}

class ChildClass extends BaseClass {
}
// Results in Fatal error: Class ChildClass may not inherit from final class
(BaseClass)

?>

```

4. Polymorphism

- Polymorphism is a Greek word that literally means many forms. In object-oriented programming, polymorphism is closely related to inheritance.
- Generally, it is of two types: compile-time (overloading) and run time (overriding), but polymorphism in PHP does not support overloading, or in other words, compile-time polymorphism.
- To implement polymorphism in PHP, you can use either abstract classes or interfaces.
- Types of Polymorphism :
 - **Runtime polymorphism**
 - **Polymorphism using Interface**
 - **Polymorphism using abstract classes**
- **Runtime polymorphism**
 - The Runtime polymorphism implies that a choice is made at runtime (not at the time of compilation), or we can have various subtypes carried out for a superclass. Function overloading is an illustration of runtime polymorphism.

5. Interface

- You can consider the interface as a blueprint for all the classes that extend it. They are similar to classes with just a single primary difference.
- You can declare and define method names and arguments within interfaces, but you cannot write method code.
- Any class implementing an interface will implement all its methods.
- Interfaces are declared with the interface keyword:
- The syntax of an interface is:

```
<?php
```

```

interface MyInterfaceName {

    public function methodA();

    public function methodB();

}

?>

```

- To implement an interface, use the implements operator as follows:

```

<?php
class MyClassName implements MyInterfaceName
{
    public function methodA()
    {

        // method A implementation

    }
    public function methodB()
    {

        // method B implementation

    }
}

?>

```

- Ex1: (Without Argument Interface Method)

```

<?php
interface MyInterfaceName
{
    public function method1();
    public function method2();
}
class MyClassName implements MyInterfaceName
{
    public function method1()
    {
        echo "Method1 Called" . "\n";
    }
}

```

```

        public function method2()
        {
            echo "Method2 Called". "\n";
        }
    }
    $obj = new MyClassName;
    $obj->method1();
    $obj->method2();
?>

```

- Ex2 (With Argument Interface Method)

```

<?php
interface demointerface
{
    public function sum($a,$b);
    public function sub($a,$b);
    public function mul($a,$b);
    public function div($a,$b);
}
class MyClassName implements demointerface
{
    public function sum($a,$b)
    {
        echo "Addition is: ". $a+$b . "<br>";
    }
    public function sub($a,$b)
    {
        echo "Subtraction is: ". $a-$b . "<br>";
    }
    public function mul($a,$b)
    {
        echo "Multiplication is: ". $a*$b . "<br>";
    }
    public function div($a,$b)
    {
        echo "Division is: ". $a/$b . "<br>";
    }
}
$obj = new MyClassName;

```

```

$obj->sum(10,5);
$obj->sub(10,5);
$obj->mul(10,5);
$obj->div(10,5);
?>

```

6. Abstract classes & methods

- PHP has abstract classes and methods.
- Any class that contains at least one abstract method must also be abstract. Methods defined as abstract simply declare the method's signature; they cannot define the implementation.
- When inheriting from an abstract class, all methods marked abstract in the parent's class declaration must be defined by the child class, and follow the usual inheritance and signature compatibility rules.
- When inheriting from an abstract class, the child class method must be defined with the same name, and the same or a less restricted access modifier.
- So, if the abstract method is defined as protected, the child class method must be defined as either protected or public, but not private.
- Also, the type and number of required arguments must be the same. However, the child classes may have optional arguments in addition.
- So, when a child class is inherited from an abstract class, we have the following rules:
 1. The child class method must be defined with the same name and it re-declares the parent abstract method.
 2. The child class method must be defined with the same or a less restricted access modifier.
 3. The number of required arguments must be the same. However, the child class may have optional arguments in addition.
- Ex 1:(Without Argument Abstract Method)


```

<?php
    // Abstract class
    abstract class Base {
        // This is abstract function
        abstract function printdata();
    }
    class Derived extends base {
        function printdata() {

```

```

        echo "Derived class";
    }
}
$b1 = new Derived;
$b1->printdata();
?>

```

- Ex 2: (With Argument Abstract Method)

```

<?php
abstract class AbstractClass
{
    // Force Extending class to define this method
    abstract protected function getValue();
    abstract protected function prefixValue($prefix);

    // Common method
    public function printOut() {
        print $this->getValue() . "<br>";
    }
}

class child1 extends AbstractClass
{
    protected function getValue() {
        return "child1";
    }

    public function prefixValue($prefix) {
        return "{$prefix}child1";
    }
}

class child2 extends AbstractClass
{
    public function getValue() {
        return "child2";
    }

    public function prefixValue($prefix) {
        return "{$prefix}child2";
    }
}

```

```

$class1 = new child1;
$class1->printOut();
echo $class1->prefixValue('abc') . "<br>";

```

```

$class2 = new child2;
$class2->printOut();
echo $class2->prefixValue('xyz') . "<br>";

```

```
?>
```

7. Trait:

- PHP implements a way to reuse code called Traits.
- Traits are a mechanism for code reuse in single inheritance languages such as PHP.
- A Trait is intended to reduce some limitations of single inheritance by enabling a developer to reuse sets of methods freely in several independent classes living in different class hierarchies.
- The semantics of the combination of Traits and classes is defined in a way which reduces complexity, and avoids the typical problems associated with multiple inheritance.

- Syntax:

- To create a trait class:

```

<?php
    trait TraitName {
        // some code...
    }

```

```
?>
```

- To access a trait class:

```

<?php
    class MyNewClass {
        use TraitName;
    }

```

```
?>
```

- Ex:

```

<?php
    trait message1 {
        public function msg1() {
            echo "OOP is fun! ";
        }
    }

```

```
trait message2 {  
    public function msg2() {  
        echo "OOP reduces code duplication!";  
    }  
}  
  
class Welcome {  
    use message1;  
}  
  
class Welcome2 {  
    use message1, message2;  
}  
  
$obj = new Welcome();  
$obj->msg1();  
echo "<br>";  
  
$obj2 = new Welcome2();  
$obj2->msg1();  
$obj2->msg2();  
?>
```

8. Magic Methods

- PHP magic methods are special methods in a class. The magic methods override the default actions when the object performs the actions.
- By convention, the names of magic methods start with a double underscore (__). And PHP reserves the methods whose names start with a double underscore (__) for magic methods.
- you have learned that the constructor and destructor use the `__construct()` and `__destruct()` methods. In fact, the constructor and destructor are also magic methods.
- Following magical methods are currently available in PHP.
- **`__call($name, $parameters)`:**
 - This method gets called when a method or property is called which has not been defined.
 - This method takes two parameters:
 - `$name`: This contains the name of the method which was called.

- `$parameters`: This is an array of parameters that were given to that method.

```
<?php
class MagicMethod
{
    function __call($name , $parameters){
        echo "Name of method =>" . $name."\n";
        echo "Parameters provided\n";
        print_r($parameters);
    }
}
// Instantiating MagicMethod
$obj = new MagicMethod();
$obj->hello("Magic" , "Method");
?>
```

- **`__toString`**

- `public __toString ( void ) : string`
- The `__toString()` method describes string representation of object. For example, what `echo $obj;` will print. This method must return a string

```
<?php
class MagicMethod {
    function __toString()
    {
        return "You are using MagicMethod object as a
String ";
    }
}
$obj = new MagicMethod();
echo $obj;
?>
```

- **`__invoke`**

- `__invoke ([ $... ] ) : mixed`
- This method is called when a script tries to call an object as a function.

```
<?php
class Employee {
    private $n;
    private $mail;
    public function __construct($n, $mail){
        $this->n = $n;
        $this->mail = $mail;
    }
}
```



```

    }
    public function __invoke(){
        echo 'Employee object when called as a
function, invokes this method for displaying some value';
    }
}
$obj3 = new Employee('Karlos','karliris@gmail.com');
// class instance used as function call
$obj3();

```

?>

- **__set**

- public __set (string \$name, mixed \$value) : void
- __set() is run when writing data to inaccessible (protected or private) or non-existing properties.

```

<?php
class MagicMethod {
    function __set($name , $value) {
        echo "You are trying to modify '"
        . $name . "' with '" . $value .
        "' which is either inaccessible
        or non-existing member";
    }
}
$obj = new MagicMethod();
$obj->value = "Hello";

```

?>

- **__get**

- public __get (string \$name) : mixed
- __get() is utilized for reading data from inaccessible (protected or private) or non-existing properties.

```

<?php
class MagicMethod {
    function __get($name){
        echo "You are trying to get '" . $name .
        " which is either inaccessible or non existing member";
    }
}
$obj = new MagicMethod();

```

```
$obj->value;
```

```
?>
```

- **__isset**

- public __isset (string \$name) : bool
- __isset() is triggered by calling isset() or empty() on inaccessible (protected or private) or non-existing properties.

```
<?php
```

```
class Manager{

    private $dat = array();

    public function __isset($n){

        echo "__isset() magic method invoked.";

        return isset($this->dat[$n]);

    }

}
```

```
$obj2 = new Manager();

echo isset($obj2->empid);
```

```
?>
```

- **__unset**

- public __unset (string \$name) : void
- __unset() is invoked when unset() is used on inaccessible (protected or private) or non-existing properties.

```
<?php
```

```
class Manager{

    private $dat = array();

    public function __isset($n)

    {

        return isset($this->dat[$n]);

    }

    public function __unset($n)

    {

        echo "__unset() magic method invoked.";

        unset($this->dat[$n]);

    }

}
```

```

    }
}
$obj2 = new Manager();
$obj2->a = 62;
if (isset($obj2->a))
    unset($obj2->a);
echo "Destroyed obj2 -> a ";
?>

```

9. Magic Constants

- ✓ Magic constants are the predefined constants in PHP which is used on the basis of their use. These constants are created by various extensions. There are nine magic constant in the PHP and all of the constant resolved at the compile-time, not like the regular constant which is resolved at run time.

- There are eight magic constants that start and end with double underscores (__).

All the constants are listed below with the example code:

1. __line__: This magic constant return the current line number of the file. If you use this magic constant in your program file somewhere then this constant will display the line number during compile time.

Syntax:

```
__line__
```

2. __file__: This magic constant return the full path of the executed file with the name of the file.

Syntax:

```
__file__
```

3. __dir__: This magic constant return the directory of the executed file. Syntax:

```
__dir__
```

4. __function__: This magic constant return the name of the function where this magic constant is included.

Syntax:

```
__function__
```

Ex:

```
echo "<br>The Line number is : ". __line__;

echo "<br>The file name is : ". __file__;

echo "<br>The directory is : ". __dir__;

function demo(){

echo "<br>The function name is : ". __function__;

}

demo();
```

5. __class__: This magic constant returns the name of the class where this magic constant is included.

Syntax:

`__class__`

10. Encapsulation

- The wrapping up of data and methods into a single unit (called class) is known as encapsulation.
- Encapsulation is a protection mechanism for the data members and methods present inside the class.
- In the encapsulation technique, we are restricting the data members from access to outside world end-users.
- In PHP, encapsulation is utilized to make the code more secure and robust.
- Using encapsulation, we are hiding the real implementation of data from the user and also does not allow anyone to manipulate data members except by calling the desired operation.
- Ex:

```
<?php
class ATM {
    private $custid;
    private $atmpin;
    public function PinChange($cid,$apin) {
        echo "-----perform tasks-----";
    }
    public function CheckBalance($cid,$apin){
```

```
        echo "Your Account Number is: $cid and your pin is: $apin";
    }
    public function miniStatement($cid) {
        echo "<br>-----perform tasks-----";
    }
}
$obj = new ATM();
$obj->CheckBalance(10005285637,"1**3");
?>
```

Unit -3 Advanced PHP

1. Predefined Variables

- PHP provides a large number of predefined variables to all scripts.
- The variables represent everything from external variables to built-in environment variables, last error messages to last retrieved headers.
- \$GLOBALS — References all variables available in global scope
- \$ _GET — HTTP GET variables
- \$ _POST — HTTP POST variables
- \$ _FILES — HTTP File Upload variables
- \$ _REQUEST — HTTP Request variables
- \$ _SESSION — Session variables
- \$ _ENV — Environment variables
- \$ _COOKIE — HTTP Cookies
- \$argc — The number of arguments passed to script
- \$argv — Array of arguments passed to script

```
<?php
echo 'My username is ' . $_ENV["USER"] . '!';
?>
```

```
<?php
echo "Number of Arguments are: ".$argc."\n";
print_r($argv);
?>
```

2. Autoloading Classes

- In order to use a class defined in another PHP script, we can incorporate it with include or require statements.
- The PHP autoloading feature allows you to load class files automatically in your program.
- PHP calls an autoloader function when you reference a class, and the autoloader function tries to figure out which file that class is in.
- Say you had a complex object-oriented PHP project with more than a hundred PHP classes. You'd need to ensure all your classes were loaded before using them.
- spl_autoload_register() function in its SPL library which allows you to register multiple autoload functions.
- It is good practice to keep each PHP class in a separate file. Also, the name of the class should be the same as the file name. For example, the Contact.php file should contain the Contact class.
- Ex:

```
<?php
spl_autoload_register(function ($class_name)
```

```

    {
        include $class_name . '.php';
    });
    $obj = new ATM();
    $obj2 = new myclass();
    echo "<br>objects of test1 and test2 class created successfully<br>";
    $obj->PinChange(10005285637,"1**3");
?>

```

3. Anonymous Classes

- As the name suggests, an anonymous class is the one which doesn't have a name. It is meant for one time use, and if one needs to define a class on the fly.
- Anonymous class can perform everything that a normal class can i.e. extend another class, implement interface or use traits.
- Definition of anonymous class lies inside an expression whose result is an object of that class. It is defined with new class syntax as follows.
- An anonymous class can do many things that a named class does, such as implementing an interface or extending a class.
- It can be useful when you only create one object instance of a class, decreasing the amount of code you write and making it easier to read.
- Also, anonymous classes can be used to avoid naming conflicts. Because they have no name, they cannot clash with other classes with the same name in your codebase.
- Syntax

```

<?php
    $obj=new class {
        public function sayhello()
        {
            echo "Hello World";
        }
    };
    $obj->sayhello();
?>

```

4. Exceptions & Error Handling

- Error handling is the process of catching errors raised by your program and then taking appropriate action.
- While writing your PHP program you should check all possible error conditions before going ahead and take appropriate action when required.
- **Using die() function**

```

<?php

```

```

if(!file_exists("/tmp/test.txt")) {
    die("File not found");
}else {
    $file = fopen("/tmp/test.txt","r");
    print "Open file successfully";
}
// Test of the code here.

```

?>

- **Try, throw and catch**

- Proper exception code should include:

1. **try** - A function using an exception should be in a "try" block. If the exception does not trigger, the code will continue as normal. However if the exception triggers, an exception is "thrown"
2. **throw** - This is how you trigger an exception. Each "throw" must have at least one "catch"
3. **catch** - A "catch" block retrieves an exception and creates an object containing the exception information.
4. **Finally** - : It is used in place of a catch block or after a catch block. Basically it is put for cleanup activity in PHP code.

```

<?php
//create function with an exception
function checkNum($number) {
    if($number>1) {
        throw new Exception("Value must be 1 or below");
    }
    return true;
}

//trigger exception in a "try" block
try {
    checkNum(2);
    //If the exception is thrown, this text will not be shown
    echo 'If you see this, the number is 1 or below';
}

//catch exception
catch(Exception $e) {
    echo 'Message: ' . $e->getMessage();
}

```



```
    }
    ?>
```

5. Object Iteration

- In PHP, object iteration refers to the process of looping through the properties and values of an object. To iterate through the properties of an object, you can use either foreach or get_object_vars() function. Here's how you can do it:

```
<?php
class MyClass
{
    public $var1 = 'value 1';
    public $var2 = 'value 2';
    public $var3 = 'value 3';
    protected $protected = 'protected var';
    private $private = 'private var';
    function iterateVisible() {
        echo "From the Class iterateVisible: <br>";
        foreach ($this as $key => $value) {
            print "$key => $value <br>";
        }
    }
}
$obj = new MyClass();
foreach($obj as $key => $value) {
    print "$key => $value <br>";
}
echo "<br>";
$obj->iterateVisible();
?>
```

6. Object Cloning, Comparing Objects

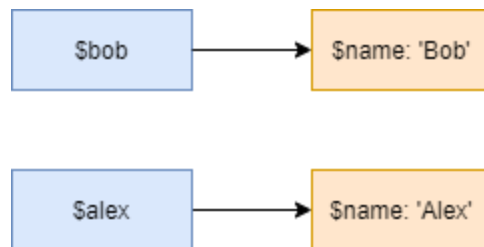
6.1 Object Cloning:

- In PHP, object cloning refers to the process of creating a duplicate of an existing object. The clone keyword is used to perform object cloning. When you clone an object, a new object is created with the same properties and values as the original object.
- The new object is entirely different from the original object, and changes made to one object will not affect the other. This behavior is often referred to as "passing by value."

- When you use the assignment operator to copy an object, you are not actually creating a new copy of the object; instead, you are creating a reference to the same object.
- In PHP, there are two types of cloning when it comes to object cloning: shallow copy and deep copy.

1. Shallow Copy:

- Shallow copy is the default behavior of object cloning in PHP when you use the clone keyword. A shallow copy creates a new object that is a duplicate of the original object, but it only copies the properties by value. It does not create new copies of any objects nested within the original object.



- Ex:

```
class MyClass {
    public $data;
}

$obj1 = new MyClass();
$obj1->data = 42;

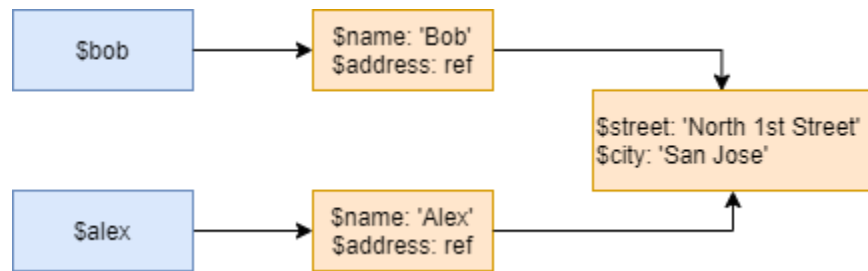
$obj2 = clone $obj1;

$obj2->data = 100;

echo $obj1->data; // Output: 42
echo $obj2->data; // Output: 100
```

2. Deep Copy:

- A deep copy, as the name implies, creates a complete copy of the original object along with all the nested objects within it.
- This is useful when you want to avoid sharing references and ensure that the original and cloned objects are entirely independent of each other. Achieving a deep copy requires implementing the `__clone()` magic method in your class and manually handling the cloning process for any nested objects.



○ Ex:

```
<?php
```

```
class myclass
```

```
{
```

```
    public $var1;
```

```
    public $var2;
```

```
    public function __clone()
```

```
    {
```

```
        $this->var1= $this->var1;
```

```
        $this->var2= $this->var2;
```

```
    }
```

```
}
```

```
$obj1=new myclass();
```

```
$val=10;
```

```
$val2=20;
```

```
$obj1->var1=&$val;
```

```
$obj1->var2=&$val2;
```

```
$obj2=clone $obj1;
```

```
$obj1->var1=50;
```

```
echo "<br>".$obj1->var1."<br>".$obj1->var2;
```

```

echo "<br>Value of second object";

echo "<br>".$obj2->var1."<br>".$obj2->var2;

?>

```

6.2 Comparing Objects:

- When comparing objects in PHP, you need to be aware of the difference between object identity comparison and object value comparison.

Object Identity Comparison:

- Object identity comparison checks whether two variables refer to the same instance of an object.
- It evaluates to true only if both variables point to the same memory location (i.e., they are referencing the exact same object).
- Ex:

```

class MyClass {}

$obj1 = new MyClass();
$obj2 = $obj1; // Both $obj1 and $obj2 point to the same object

var_dump($obj1 === $obj2); // Output: true

```

- **Object Value Comparison:**
- Object value comparison checks whether the properties and their values of two objects are the same.
- It evaluates to true if the objects have the same property values, even if they are different instances.
- Ex:

```

class MyClass {
    public $data;
}

$obj1 = new MyClass();
$obj1->data = 42;

$obj2 = new MyClass();
$obj2->data = 42; // Both $obj1 and $obj2 have the same property value

// Comparing object values manually
$areEqual = ($obj1->data === $obj2->data);

```

```
var_dump($areEqual); // Output: true
```

7. Type Hinting

- Type hinting in PHP is a feature that allows you to specify the data type of a parameter in a function or method declaration.
- It ensures that the function can only accept arguments of the specified data type, making the code more reliable and easier to maintain.
- Type hinting can be used for the following data types:
- Scalar types:
 - int: Specifies that the parameter should be an integer.
 - float: Specifies that the parameter should be a floating-point number.
 - string: Specifies that the parameter should be a string.
 - bool: Specifies that the parameter should be a boolean value.
- Ex:

```
<?php
    function div(int $a,int $b): int
    {
        return ($a/$b);
    }
    echo div(10,3);
?>
```

8. Objects and References

- A PHP reference is an alias, which allows two different variables to write to the same value. In PHP, an object variable doesn't contain the object itself as value.
- It only contains an object identifier which allows object accessors to find the actual object.
- When an object is sent by argument, returned or assigned to another variable, the different variables are not aliases: they hold a copy of the identifier, which points to the same object.
- ex:

```
<?php
class A {
    public $a = 1;
}
$a = new A;
$b = $a; // $a and $b are copies of the same identifier
        // ($a) = ($b) = <id>
echo "Value of variable is before change in reference variable : ".$a->a."<br>";
$b->a = 2;
echo "Value of variable is after change in reference variable : ".$a->a."<br>";
$c = new A;
$d = &$c; // $c and $d are references
```

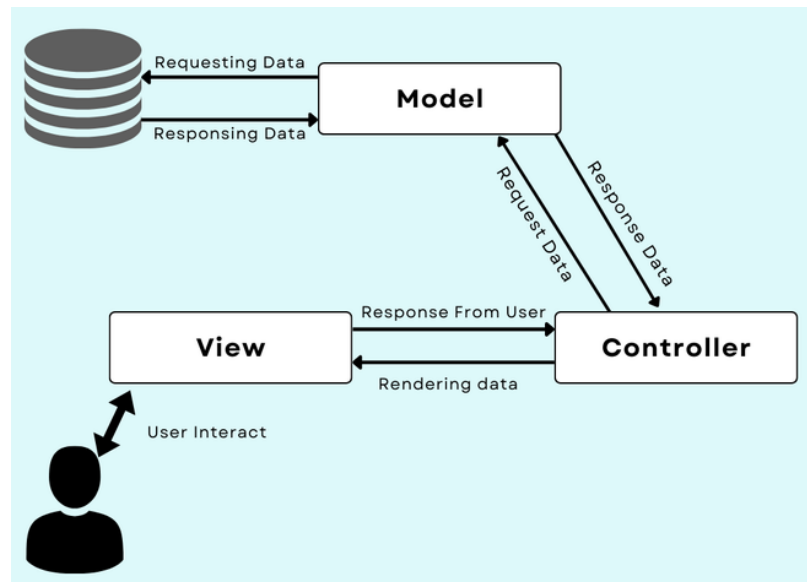
```
// ($c,$d) = <id>
$d->a = 3;
echo "Value of variable is: ".$c->a."<br>";

?>
```

Unit - 4 Introduction to MVC using Laravel

1. What is MVC? and Explain MVC Architecture

- MVC stands for "Model view And Controller".
- The main aim of MVC Architecture is to separate the Business logic & Application data from the USER interface.
- Different types of Architectures are available. These are 3-tier Architecture, N-tier Architecture, MVC Architecture, etc.



- In the above image, we can understand how the MVC framework interacts with the User and the Database.
- **Model** –
 - This component of the MVC framework handles data used in your application.
 - It helps to retrieve the data from the **database** and then perform some operation that your application is supposed to perform then it stores that data back in the database.
 - In simple words, we can say that Model is responsible for managing data that is passed between the database and the User Interface (**View**).
- **Controller** –
 - This controller component helps to interact with the model component to fetch data from the database and then pass that data to the view component to get the desired output on the user's browser screen.
 - Same way, when the user enters some data the controller fetches that data and then performs some operation or just inserts that data into the database with the use of the model components.
- **Views** –

- This component is User Interface, which defines the template which is sent as a response to the browser.
- This View components contain the part of code which helps to display data to the User Interface on the user's browser.
- For example, we can say the buttons, textbox, dropdown menu, and many more such widgets on the browser screen are the part of View Component.

2. What is Laravel ? Explain Features of Laravel.

- Laravel is an open-source PHP framework, which is robust and easy to understand. It follows a model-view-controller design pattern.
- Laravel reuses the existing components of different frameworks which helps in creating a web application. The web application thus designed is more structured.
- If you are familiar with Core PHP and Advanced PHP, Laravel will make your task easier. It saves a lot of time.
- Moreover, a website built in Laravel is secure and prevents several web attacks.
- **Features of Laravel**
- Laravel offers the following key features which makes it an ideal choice for designing web applications –
- **Modularity**
- Laravel provides 20 built in libraries and modules which helps in enhancement of the application. Every module is integrated with Composer dependency manager which eases updates.
- **Testability**
- Laravel includes features and helpers which helps in testing through various test cases. This feature helps in maintaining the code as per the requirements.
- **Routing**
- Laravel provides a flexible approach to the user to define routes in the web application. Routing helps to scale the application in a better way and increases its performance.
- **Configuration Management**
- A web application designed in Laravel will be running on different environments, which means that there will be a constant change in its configuration. Laravel provides a consistent approach to handle the configuration in an efficient way.
- **Query Builder and ORM**
- Laravel incorporates a query builder which helps in querying databases using various simple chain methods. It provides ORM (Object Relational Mapper) and ActiveRecord implementation called Eloquent.
- **Template Engine**
- Laravel uses the Blade Template engine, a lightweight template language used to design hierarchical blocks and layouts with predefined blocks that include dynamic content.
- **E-mail**

- Laravel includes a mail class which helps in sending mail with rich content and attachments from the web application.
- **Authentication**
- User authentication is a common feature in web applications. Laravel eases designing authentication as it includes features such as register, forgot password and send password reminders.

3. Installation of laravel

a. Basic Requirements for Laravel

- The Laravel framework has a few system requirements. All of these requirements are satisfied by the Laravel Homestead virtual machine, so it's highly recommended that you use Homestead as your local Laravel development environment.
- However, if you are not using Homestead, you will need to make sure your server meets the following requirements:

TOOL	PURPOSE
PHP (V8.1 OR HIGHER)	Laravel runs on PHP
COMPOSER	A tool to install Laravel and PHP packages
WEB SERVER	Apache or Nginx (usually comes with XAMPP or Laragon)
DATABASE	MySQL or MariaDB (used to store app data)
TEXT EDITOR/IDE	VS Code, Sublime Text, etc.

b. Use of Composer, Laravel Install Using Composer,

- For managing dependencies, Laravel uses composer. Make sure you have a Composer installed on your system before you install Laravel. In this chapter, you will see the installation process of Laravel.
- You will have to follow the steps given below for installing Laravel onto your system –
- **Step 1** – Visit the following URL and download composer to install it on your system.
- <https://getcomposer.org/download/>
- **Step 2** – After the Composer is installed, check the installation by typing the Composer command in the command prompt as shown in the following screenshot.

```

Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\wamp\www\laravel>php artisan --version
Laravel Framework version 5.1.23 (LTS)

C:\wamp\www\laravel>cd\

C:\>composer

Composer version 1.0-dev (c7ed232ef42c2bd63cd8a057b6c7c8043b37cd5a) 2015-10-29 09:52:59

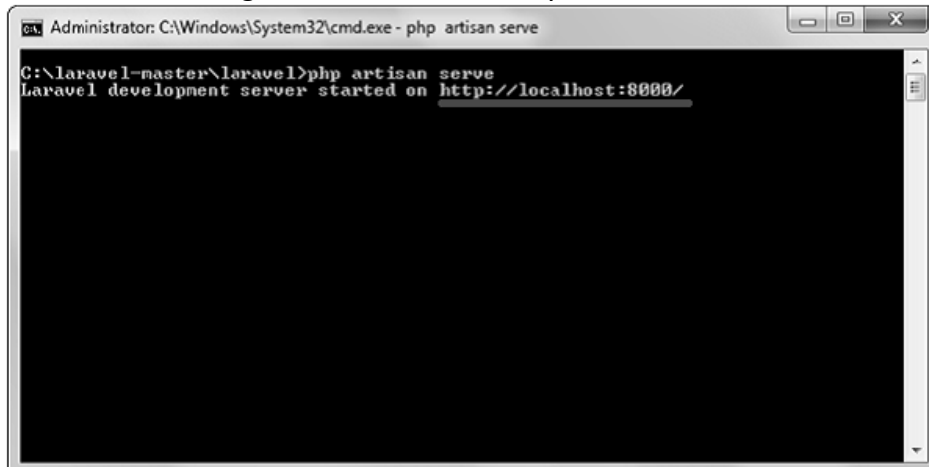
Usage:
  command [options] [arguments]

Options:
  -h, --help                Display this help message
  -q, --quiet               Do not output any message
  -V, --version             Display this application version
  --ansi                   Force ANSI output
  --no-ansi                Disable ANSI output
  -n, --no-interaction      Do not ask any interactive question
  --profile                Display timing and memory usage information
  -d, --working-dir=WORKING-DIR If specified, use the given directory as working directory.
  --verbose                Increase the verbosity of messages: 1 for normal output, 2 for more verbose output and 3 for debug
  
```

- **Step 3** – Create a new directory anywhere in your system for your new Laravel project. After that, move to path where you have created the new directory and type the following command there to install Laravel.
- `composer create-project laravel/laravel --prefer-dist`
- Now, we will focus on installation of version 5.7. In Laravel version 5.7, you can install the complete framework by typing the following command –
- `composer create-project laravel/laravel project1`
- The output of the command is as shown below –

```
→ code composer create-project laravel/laravel test dev-develop
Installing laravel/laravel (dev-develop d6acad21cb2288713d9c09a31f9b4ab86f116039)
- Installing laravel/laravel (dev-develop develop): Cloning develop from cache
Created project in test
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies (including require-dev)
Package operations: 71 installs, 0 updates, 0 removals
- Installing vlucas/phpdotenv (v2.5.1): Loading from cache
- Installing symfony/css-selector (v4.1.3): Loading from cache
- Installing tijsverkoyen/css-to-inline-styles (2.2.1): Loading from cache
- Installing symfony/polyfill-php72 (v1.9.0): Loading from cache
- Installing symfony/polyfill-mbstring (v1.9.0): Loading from cache
- Installing symfony/var-dumper (v4.1.3): Loading from cache
- Installing symfony/routing (v4.1.3): Loading from cache
- Installing symfony/process (v4.1.3): Loading from cache
- Installing symfony/polyfill-ctype (v1.9.0): Loading from cache
- Installing symfony/http-foundation (v4.1.3): Loading from cache
- Installing symfony/event-dispatcher (v4.1.3): Loading from cache
- Installing psr/log (1.0.2): Loading from cache
- Installing symfony/debug (v4.1.3): Loading from cache
- Installing symfony/http-kernel (v4.1.3): Loading from cache
- Installing paragonie/random_compat (v9.99.99): Loading from cache
```

- The Laravel framework can be directly installed with develop branch which includes the latest framework.
- **Step 4** – The above command will install Laravel in the current directory. Start the Laravel service by executing the following command.
- `php artisan serve`
- **Step 5** – After executing the above command, you will see a screen as shown below –



- **Step 6** – Copy the URL underlined in gray in the above screenshot and open that URL in the browser. If you see the following screen, it implies Laravel has been installed successfully.

4. Configuration of laravel

a. Introduction

- Typically, you will need to publish your package's configuration file to the application's config directory. This will allow users of your package to easily override your default configuration options. To allow your configuration files to be published, call the publishes method from the boot method of your service provider:

b. Environment Configuration

- It is often helpful to have different configuration values based on the environment where the application is running. For example, you may wish to use a different cache driver locally than you do on your production server.
- In a fresh Laravel installation, the root directory of your application will contain a **.env.example** file. If you install Laravel via Composer, this file will automatically be copied to **.env**. Otherwise, you should copy the file manually.
- Your **.env** file should not be committed to your application's source control, since each developer / server using your application could require a different environment configuration.
- If you are developing with a team, you may wish to continue including a **.env.example** file with your application. By putting placeholder values in the example configuration file, other developers on your team can clearly see which environment variables are needed to run your application. You may also create a **.env.testing** file. This file will override the **.env** file when running PHPUnit tests or executing Artisan commands with the **--env=testing** option.

c. Database Configuration

- Laravel makes connecting with databases and running queries extremely simple. The database configuration file is **app/config/database.php**.
- In this file you may define all of your database connections, as well as specify which connection should be used by default.
- Examples for all of the supported database systems are provided in this file.
- Currently Laravel supports four database systems: MySQL, Postgres, SQLite, and SQL Server.

5. Web application Security Challenges (SQL injection, XSS, CSRF)

- **CSRF: (Cross-Site Request Forgery)**
- Cross-Site Request Forgery (CSRF) is a type of attack that is performed by the attacker to send requests to a system with the help of an authorized user who is trusted by the system.
- Laravel provides protection with the CSRF attacks by generating a CSRF token. This CSRF token is generated automatically for each user. This token is nothing but a random string that is managed by the Laravel application to verify the user requests.
- How to Use: This CSRF token protection can be applied to any HTML form in Laravel application by specifying a hidden form field of CSRF token. The requests are validated automatically by the CSRF VerifyCsrfToken middleware.

- There are three different ways in which you can do this.
 1. @csrf
 2. csrf_field()
 3. csrf_token()
- @csrf: This is a blade template directive for generating the hidden input field in the HTML form.
- Syntax:


```
<form method="POST">
    @csrf // Generate hidden input field
    .....
    .....
</form>
```
- Example:


```
<!DOCTYPE html>
<html>
<head>
    <title>Laravel | CSRF Protection</title>
</head>
<body>
    <section>
        <h1>CSRF Protected HTML Form</h1>
        <form method="POST">
            @csrf

            <input type="text" name="username"
                placeholder="Username">
            <input type="password" name="password"
                placeholder="Password">
            <input type="submit" name="submit" value="Submit">
        </form>
    </section>
</body>
</html>
```
- csrf_field(): This function can be used to generate the hidden input field in the HTML form.
- Note: This function should be written inside double curly braces.
- Syntax:


```
<form method="POST">
    // Generate hidden input field
    {{ csrf_field() }}
    .....
</form>
```
- Example:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Laravel | CSRF Protection</title>
  </head>
  <body>
    <h1>CSRF Protected HTML Form</h1>
    <form method="POST">
      {{ csrf_field() }}

      <input type="text" name="username"
            placeholder="Username">
      <input type="password" name="password"
            placeholder="Password">
      <input type="submit" name="submit"
            value="Submit">

    </form>
  </body>
</html>

```

- `csrf_token()`: This function just gives a random string. This function does not generate the hidden input field.
- Note: HTML input field should be written explicitly. This function should be written inside double curly braces.

- Syntax:

```

<form method="POST">
  <input type="hidden" name="_token" value="{{ csrf_token() }}">
  .....
  .....
</form>

```

- Example:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Laravel | CSRF Protection</title>
  </head>
  <body>
    <section>
      <h1>CSRF Protected HTML Form</h1>
      <form method="POST">
        <input type="hidden" name="_token" value="{{ csrf_token() }}">

        <input type="text" name="username"

```

```

        placeholder="Username">
        <input type="password" name="password"
        placeholder="Password">
        <input type="submit" name="submit"
        value="Submit">
    </form>
</section>
</body>
</html>

```

○ **SQL INJECTION:**

- Laravel is a free open-source PHP framework. It follows the MVC design pattern and has built-in tools for performing tasks like user authentication, routing, and database operations.
- With the help of the Eloquent ORM, you can build a small Laravel app that reads and writes data to an SQL database without writing a single raw SQL query.
- That is, you don't need to write queries like "SELECT * FROM table" to read data from SQL. However, Laravel supports raw SQL query, as your desired task may require raw queries in some cases.
- Laravel does this by making secured and prepared statements that are going to escape any user input that may come in through the apps. If attackers add a new input to a form, they may try to insert a query and then run their own SQL query to damage your application database. However, this won't work while you are using Eloquent. Eloquent is going to protect from this SQL query and the invalid query will just be saved as text into your database.
- Laravel's Eloquent ORM uses PDO binding that protects web apps from SQL injections. This relevant feature ensures that no client could modify the intent of the SQL queries.
- Consider the example of the form used to collect users' email addresses from a database. the form will search for an email address, for instance, "abc@example.com". Now imagine that the SQL query is modified to:
- **SELECT * FROM users WHERE email = 'abc@example.com' or 1=1**
- In the above example, 1=1 is a simple expression that always evaluates to be true. If it is attached to the above query with the OR condition, the query will fetch all records from the table because the SELECT condition will evolve to be always true.
- This framework provides other ways of talking to databases, such as raw SQL queries. Yet, Eloquent remains one of the most popular options.
- Laravel framework uses PDO binding to prevent SQL injection attacks because no variable gets to the database without validation.
- If this is the case, one should always use prepared SQL queries to prevent misplacement. Consider the following statement that looks ripe for SQL injection:

```

Route::get('this-is-prone-to-sql-injection', function() {
    $name = "'ancy' OR 1=1";
    return DB::select(

```

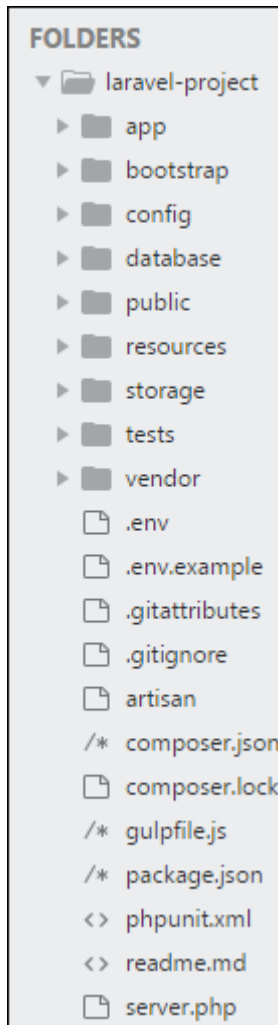
- ```
DB::raw("SELECT * FROM users WHERE name = $name"));});
```
- Here the statement `1=1` used in the OR condition will result in returning all the rows in the user's table. This can be prevented by using the following code instead:

```
Route::get('safe-from-sql-injection', function() {
 $name = "'ancy' OR 1=1";
 return DB::select(
 DB::raw("SELECT * FROM users WHERE name = ?", [$name]));});
```
  - Laravel replaces the question marks with the query, automatically escaping the input variables. This protects the query from SQL injection attacks.
  - **Xss:(Cross-Site Scripting)**
  - "XSS occurs whenever an application includes untrusted data in a new web page without proper validation or escaping, or updates an existing web page with user-supplied data using a browser API that can create HTML or JavaScript.
  - XSS allows attackers to execute scripts in the victim's browser which can hijack user sessions, deface web sites, or redirect the user to malicious sites."
  - Never display user-supplied input without escaping the data.
  - Laravel's template engine, Blade, automatically escapes content rendered using the default `{{ $var }}` syntax.
  - This sends it through PHP's `htmlspecialchars` function.

## Unit -5 Laravel - MVC Application

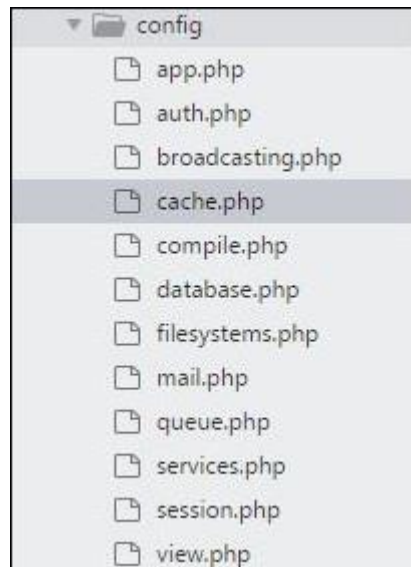
### 1. Directory Structure

- The application structure in Laravel is basically the structure of folders, sub-folders and files included in a project. Once we create a project in Laravel, we get an overview of the application structure as shown in the image here.
- The snapshot shown here refers to the root folder of Laravel, namely laravel-project. It includes various sub-folders and files. The analysis of folders and files, along with their functional aspects is given below –



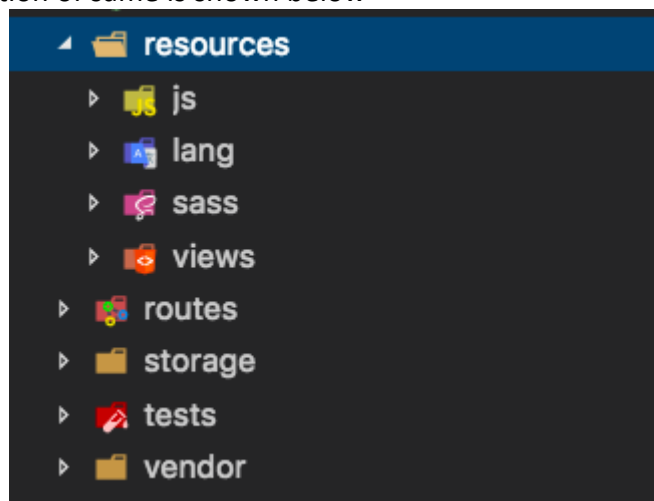


- **App**
- It is the application folder and includes the entire source code of the project. It contains events, exceptions and middleware declarations. The app folder comprises various sub folders as explained below –
- **Http**
- The Http folder has sub-folders for controllers, middleware and application requests. As Laravel follows the MVC design pattern, this folder includes model, controllers and views defined for the specific directories.
- The Middleware sub-folder includes middleware mechanism, comprising the filter mechanism and communication between response and request.
- The Requests sub-folder includes all the requests of the application.
- **Bootstrap**
- This folder encloses all the application bootstrap scripts. It contains a sub-folder namely cache, which includes all the files associated for caching a web application. You can also find the file app.php, which initializes the scripts necessary for bootstrap.
- **Config**
- The config folder includes various configurations and associated parameters required for the smooth functioning of a Laravel application. Various files included within the config folder are as shown in the image here. The filenames work as per the functionality associated with them.



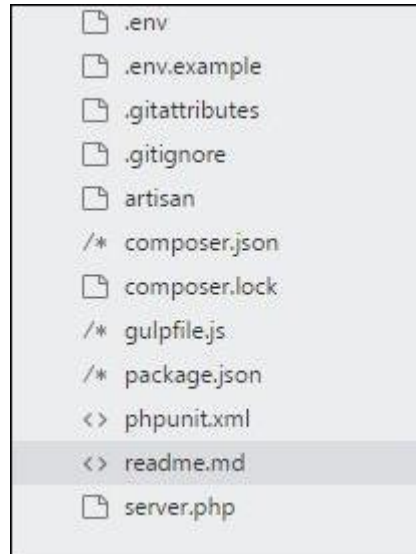
- **Database**
- As the name suggests, this directory includes various parameters for database functionalities. It includes three sub-directories as given below –
- Seeds – This contains the classes used for unit testing database.
- Migrations – This folder helps in queries for migrating the database used in the web application.
- Factories – This folder is used to generate large number of data records.

- **Public**
- It is the root folder which helps in initializing the Laravel application. It includes the following files and folders –
- .htaccess – This file gives the server configuration.
- javascript and css – These files are considered as assets.
- index.php – This file is required for the initialization of a web application.
- **Resources**
- Resources directory contains the files which enhances your web application. The sub-folders included in this directory and their purpose is explained below –
- assets – The assets folder include files such as LESS and SCSS, that are required for styling the web application.
- lang – This folder includes configuration for localization or internalization.
- views – Views are the HTML files or templates which interact with end users and play a primary role in MVC architecture.
- Observe that the resources directory will be flattened instead of having an assets folder. The pictorial representation of same is shown below –



- **Storage**
- This is the folder that stores all the logs and necessary files which are needed frequently when a Laravel project is running. The sub-folders included in this directory and their purpose is given below –
- app – This folder contains the files that are called in succession.
- framework – It contains sessions, cache and views which are called frequently.
- Logs – All exceptions and error logs are tracked in this sub folder.
- **Tests**
- All the unit test cases are included in this directory. The naming convention for naming test case classes is camel\_case and follows the convention as per the functionality of the class.

- **Vendor**
- Laravel is completely based on Composer dependencies, for example to install Laravel setup or to include third party libraries, etc. The Vendor folder includes all the composer dependencies.
- In addition to the above mentioned files, Laravel also includes some other files which play a primary role in various functionalities such as GitHub configuration, packages and third party libraries.
- The files included in the application structure are shown below –



## 2. Artisan Commands

Artisan is a command-line interface (CLI) included with the Laravel PHP framework that provides several helpful commands for speeding up development workflows.

1. For Controller: The following command will create a controller:

**php artisan make:controller STDController**

2. For Eloquent Model: The following command will create a eloquent model:

**php artisan make:model STD**

3. For Migration: The following command will create a migration:

**php artisan make:migration create\_STD\_table**

4. To do database migration for all the tables, run the command below:

**php artisan migrate**

5. **For Starting Development Server:** The following command will start the Laravel development server and provide a URL to visit the running Laravel application:

**php artisan serve**

### 3. Routing

- Route means URL of website.
- All Laravel routes are defined in route files found within the routes directory. The application's App\Providers\RouteServiceProvider automatically stacks these files. And the routes/web.php file defines the routes for your web interface.
- The default installation of Laravel comes with two routes: one for the web and the other for API. Here is what the route for web in web.php looks like:

```
Route::get('/', function () {
 return view('welcome');
});
```

- The above code defines a route for the homepage. Every time this route receives a get request for /, It will return the view welcome. Views are the frontend of the application
- The structure of the route is very simple. Open the appropriate file (either `web.php` or `api.php`) and start the code with `Route::` This is followed by the request you want to assign to that specific route and then comes the function that will be executed as a result of the request.
- We can also use the route as a direct return view.

```
Route::view('welcome', '/welcome');
```

- another method of creating route is:

```
Route::get('/', function () {
 return "<h1> Welcome Page </h1>";
});
```

### 4. Controller

- Laravel is an MVC based PHP framework. In MVC architecture, 'C' stands for 'Controller'. A Controller is that which controls the behavior of a request.
- It handles the requests coming from the Routes. In Laravel, a controller is in the 'app/Http/Controllers' directory. All the controllers, that are to be created, should be in this directory.
- We can create a controller using 'make:controller' Artisan command.
- **Syntax:**
- `php artisan make:controller UserController`
- You can specify any name in place of 'User', but according to the naming convention of Laravel, you have to specify the word 'Controller' at the end for any name you specify.

### 5. Models

- The Model corresponds to the letter 'M' in the MVC framework. The motive of Models in any Model view controller framework web application is to manage business logic.
- A Model is nothing but a class in the Laravel framework. This class is responsible to interact with underlying database tables.

- Laravel with Eloquent ORM is capable of handling the database more efficiently. ORM stands for 'Object Relational Mapper' and it is responsible for your swift interaction with the database.
- These Models provide you with an easy way to update, insert or retrieve data from the tables.
- the simplest way of model instance creation using Artisan command. like this:
- `php artisan make:model <Model-Name>`

## 6. Views

- Views contain the html code required by your application, and it is a method in Laravel that separates the controller logic and domain logic from the presentation logic. Views are located in the resources folder, and its path is resources/views.
- example to understand more about Views –
- **Step 1** – Copy the following code and save it at resources/views/test.php
 

```
<html>
 <body>
 <h1>Hello, World</h1>
 </body>
</html>
```
- **Step 2** – Add the following line in app/Http/routes.php file to set the route for the above view.
 

```
app/Http/routes.php
Route::get('/test', function() {
 return view('test');
});
```
- **Step 3** – Visit the following URL to see the output of the view.
- <http://localhost:8000/test>
- **Passing Data to Views:**
- While building application it may be required to pass data to the views.
- Pass an array to view helper function. After passing an array, we can use the key to get the value of that key in the HTML file.
- the following example to understand more about passing data to views –
- **Step 1** – Copy the following code and save it at resources/views/test.php
 

```
<html>
 <body>
 <h1><?php echo $name; ?></h1>
 </body>
</html>
```
- **Step 2** – Add the following line in app/Http/routes.php file to set the route for the above view.
 

```
app/Http/routes.php
```

```
Route::get('/test', function() {
 return view('test', ['name'=>'Virat Gandhi']);
});
```

- **Step 3** – The value of the key name will be passed to test.php file and \$name will be replaced by that value.
- **Step 4** – Visit the following URL to see the output of the view.  
http://localhost:8000/test

## 7. Database (CRUD) Operations

- **How to insert data into database in Laravel-8 using Eloquent model**
- **Step 1:** Create a Model and Migration by following command:

**\$ php artisan make:model Student -m**

- **Model:** Lets open Student Model in following path: app/Model/Student.php
- ```
<?php
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
```

```
class Student extends Model
{
    use HasFactory;
    protected $table = 'students';
    protected $fillable = [
        'name',
        'email',
        'course',
        'section',
    ];
}
```

- **Migration:** Lets open create_students_table.php Migration table in following path:
database/migrations/2021_05_30_create_students_table.php: (ADD this in your migration table)

```
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->string('course');
        $table->string('section');
        $table->timestamps();
    });
}
```

```
}
```

- Now lets migrate this students table into our database by following command:
\$ php artisan migrate
- **Step 2:** Go to web.php file in the following path as: routes/web.php and create a route (url) as follows:

```
use App\Http\Controllers\StudentController;
```

```
Route::get('add-student', [StudentController::class, 'create']);
```

```
Route::post('add-student', [StudentController::class, 'store']);
```

- **Step 3:** Create a Controller named StudentController by following command:
\$ php artisan make:controller StudentController
- after successfully creating the controller lets write the code to View the FORM and insert data from Form in Laravel as follows:

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Models\Student;
```

```
use Illuminate\Http\Request;
```

```
class StudentController extends Controller
```

```
{
```

```
    public function create()
```

```
    {
```

```
        return view('student.create');
```

```
    }
```

```
    public function store(Request $request)
```

```
    {
```

```
        $student = new Student;
```

```
        $student->name = $request->input('name');
```

```
        $student->email = $request->input('email');
```

```
        $student->course = $request->input('course');
```

```
        $student->section = $request->input('section');
```

```
        $student->save();
```

```
        return redirect()->back()->with('status','Student Added Successfully');
```

```
    }
```

```
}
```

- **Step 4:** Create a blade file named create.blade.php in a student folder as show in following the path as: resource/views/student/create.blade.php

```
<div class="container">
```

```
    <div class="row justify-content-center">
```

```

<div class="col-md-6">

    @if (session('status'))
        <h6 class="alert alert-success">{{ session('status') }}</h6>
    @endif

    <div class="card">
        <div class="card-header">
            <h4>Add Student</h4>
        </div>
        <div class="card-body">

            <form action="{{ url('add-student') }}" method="POST">
                @csrf

                <div class="form-group mb-3">
                    <label for="">Student Name</label>
                    <input type="text" name="name" class="form-control">
                </div>
                <div class="form-group mb-3">
                    <label for="">Student Email</label>
                    <input type="text" name="email" class="form-control">
                </div>
                <div class="form-group mb-3">
                    <label for="">Student Course</label>
                    <input type="text" name="course" class="form-control">
                </div>
                <div class="form-group mb-3">
                    <label for="">Student Section</label>
                    <input type="text" name="section" class="form-control">
                </div>
                <div class="form-group mb-3">
                    <button type="submit" class="btn btn-primary">Save
Student</button>
                </div>
            </form>
        </div>
    </div>
</div></div>

```


- **Step 6:** Now we are done with the code of Insert data in laravel. Let's Run the artisan command to start the application as follows:

\$ php artisan serve

- after servering the artisan - goto: <http://localhost:8000/add-student>
- **How to fetch/retrieve data from database in Laravel**
- **Step 1:** To fetch the data from database you have been already created a Model and Migration and a Controller in laravel application:
- **Step 2:** Give a route to call the function in your controller for calling the page in the following path: routes/web.php as follows:

```
<?php
```

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;
```

```
Route::get('students', [StudentController::class, 'index']);
```

- **Step 3:** Let's go to the controller in the following path :
app/Http/Controllers/StudentController.php file.

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Models\Student;
use Illuminate\Http\Request;
```

```
class StudentController extends Controller
{
    public function index()
    {
        $student = Student::all();
        return view('student.index', compact('student'));
    }
}
```

- **Step 4:** Create a blade file named index.blade.php inside student folder as the following path: resources/views/student/index.blade.php file as follows:

```
<div class="container">
    <div class="row">
        <div class="col-md-12">
            <div class="card">
                <div class="card-header">
                    <h4>How to Fetch data in Laravel 8</h4>
                </div>
```

```

<div class="card-body">

    <table class="table table-bordered table-striped">
        <thead>
            <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Email</th>
                <th>Course</th>
                <th>Section</th>
                <th>Edit</th>
                <th>Delete</th>
            </tr>
        </thead>
        <tbody>
            @foreach ($student as $item)
                <tr>
                    <td>{{ $item->id }}</td>
                    <td>{{ $item->name }}</td>
                    <td>{{ $item->email }}</td>
                    <td>{{ $item->course }}</td>
                    <td>{{ $item->section }}</td>
                    <td>
                        <a href="" class="btn btn-primary btn-sm">Edit</a>
                    </td>
                    <td>
                        <a href="" class="btn btn-danger btn-sm">Delete</a>
                    </td>
                </tr>
            @endforeach
        </tbody>
    </table>

</div>
</div>
</div>
</div>
</div>

```

- Now, you can successfully run the application by the following command:
\$ php artisan serve
- now lets go to the browser at the link: <http://localhost:8000/students>

- **How to edit and update data into database in Laravel**
- **Step 1:** Fetch data from database in index.blade.php file in the following path:
resources/views/student/index.blade.php
- Let fetch the data in HTML Table to make edit button in file as follows:

```

<div class="container">
  <div class="row">
    <div class="col-md-12">
      <div class="card">
        <div class="card-header">
          <h4>How to Edit and Update Data in Laravel
            <a href="{{ url('add-student') }}" class="btn btn-primary float-
end">Add Student</a>
          </h4>
        </div>
        <div class="card-body">

          <table class="table table-bordered table-striped">
            <thead>
              <tr>
                <th>ID</th>
                <th>Name</th>
                <th>Email</th>
                <th>Course</th>
                <th>Section</th>
                <th>Edit</th>
              </tr>
            </thead>
            <tbody>
              @foreach ($student as $item)
                <tr>
                  <td>{{ $item->id }}</td>
                  <td>{{ $item->name }}</td>
                  <td>{{ $item->email }}</td>
                  <td>{{ $item->course }}</td>
                  <td>{{ $item->course }}</td>
                  <td>
                    <a href="{{ url('edit-student/' . $item->id) }}" class="btn
btn-primary btn-sm">Edit</a>
                  </td>
                </tr>
              @endforeach
            </tbody>
          </table>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

        </table>
    </div>
</div>
</div>
</div>
</div>

```

- **Step 2:** Create a edit.blade.php file in the following path:

resources/views/student/edit.blade.php to edit and update data as follows:

```

<div class="container">
    <div class="row">
        <div class="col-md-12">
            @if (session('status'))
                <h6 class="alert alert-success">{{ session('status') }}</h6>
            @endif
            <div class="card">
                <div class="card-header">
                    <h4>Edit & Update Student
                    <a href="{{ url('students') }}" class="btn btn-danger float-
end">BACK</a>
                </h4>
            </div>
            <div class="card-body">

```

```

                <form action="{{ url('update-student/' . $student->id) }}"
method="POST">
                    @csrf
                    @method('PUT')

                    <div class="form-group mb-3">
                        <label for="">Student Name</label>
                        <input type="text" name="name" value="{{ $student-
>name }}" class="form-control">
                    </div>
                    <div class="form-group mb-3">
                        <label for="">Student Email</label>
                        <input type="text" name="email" value="{{ $student->email }}"
class="form-control">
                    </div>
                    <div class="form-group mb-3">
                        <label for="">Student Course</label>
                        <input type="text" name="course" value="{{ $student-
>course }}" class="form-control">

```

```

        </div>
        <div class="form-group mb-3">
            <label for="">Student Section</label>
            <input type="text" name="section" value="{{ $student-
>section }}" class="form-control">
        </div>
        <div class="form-group mb-3">
            <button type="submit" class="btn btn-primary">Update
Student</button>
        </div>
    </form>
</div>
</div>
</div>
</div>
</div>

```

- **Step 3:** Give a route to call the function in your controller for calling the page in the following path: routes/web.php as follows:

```
<?php
```

```

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;

```

```

Route::get('students', [StudentController::class, 'index']);
Route::get('edit-student/{id}', [StudentController::class, 'edit']);
Route::put('update-student/{id}', [StudentController::class, 'update']);

```

- **Step 4:** Let's go to the controller in the following path :
app/Http/Controllers/StudentController.php file.

```
<?php
```

```
namespace App\Http\Controllers;
```

```

use App\Models\Student;
use Illuminate\Http\Request;

```

```

class StudentController extends Controller
{
    public function index()
    {
        $student = Student::all();
        return view('student.index', compact('student'));
    }
}

```

```

public function edit($id)
{
    $student = Student::find($id);
    return view('student.edit', compact('student'));
}

public function update(Request $request, $id)
{
    $student = Student::find($id);
    $student->name = $request->input('name');
    $student->email = $request->input('email');
    $student->course = $request->input('course');
    $student->section = $request->input('section');
    $student->update();
    return redirect()->back()->with('status','Student Updated Successfully');
}
}

```

- **Step 5:** We are successfully done with Edit and Update Coding, lets test it guys by serving this application as following command:

\$ php artisan serve

- now lets go to the browser at the link: <http://localhost:8000/students>

- **How to delete data from database using Eloquent Modal**

- **Step 1:** add below html table <tr> with <a> tag in your fetched record table as follows:

```

<td>
    <a href="{{ url('delete-student/' . $item->id) }}" class="btn btn-danger btn-sm">Delete</a>
</td>

```

- **Step 2:** Create a route in the following path routes/web.php as follows:

```

<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\StudentController;

```

```

Route::delete('delete-student/{id}', [StudentController::class, 'destroy']);

```

- **Step 3:** To delete the data, paste the below code in Controller as we created in app/Http/Controllers/StudentController.php

```

<?php

namespace App\Http\Controllers;

use App\Models\Student;
use Illuminate\Http\Request;

```

```
class StudentController extends Controller
{
    public function destroy($id)
    {
        $student = Student::find($id);
        $student->delete();
        return redirect()->back()->with('status','Student Deleted Successfully');
    }
}
```